

Leme

Manual técnico

Instalação, modo interativo e modo direto, referência de cada comando, perfis, scripts e códigos de saída, console web, operação do cluster, segurança, desempenho e solução de problemas da CLI do cluster chave-valor.

Versão 1.0.0
Outubro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01 Apresentação 5
- 1.1 O que vem no projeto 5
- 1.2 Tecnologias 5
- 1.3 Estrutura de pastas 6
- 1.4 Como ler este manual 6

02 Instalação 7
- 2.1 Requisitos 7
- 2.2 Com Docker 7
- 2.3 Sem Docker 8
- 2.4 Binários prontos 8
- 2.5 Primeiro acesso 9
- 2.6 Autocompletar do shell 9

03 Conceitos 10
- 3.1 Cluster e nós 10
- 3.2 Anel e réplicas 10
- 3.3 Consistência 10
- 3.4 Versões 11
- 3.5 TTL e lápides 11
- 3.6 Dicas e reparo 11

04 Conectando ao cluster 12
- 4.1 Informando os nós 12
- 4.2 Perfis de conexão 12
- 4.3 Ordem de prioridade 13
- 4.4 Token 13
- 4.5 TLS 13

05 Modo interativo 14
- 5.1 Abrindo 14
- 5.2 Editando a linha 14
- 5.3 Valores com espaços e caracteres especiais 15
- 5.4 Comandos de sessão 15
- 5.5 Confirmações 16

06 Scripts e automação 17
- 6.1 Modo direto 17
- 6.2 Formatos de saída 17
- 6.3 Códigos de saída 17

Copyright (©) 2026 Erik Pimenta

6.4 Entrada padrão e arquivos 18
6.5 Arquivos de comandos 18
6.6 PowerShell 19

07 Referência de comandos 20
7.1 get 20
7.2 put 20
7.3 del 21
7.4 exists, ttl e expire 21
7.5 scan, keys e count 21
7.6 locate e ping 21
7.7 cluster status, nodes e ring 22
7.8 node stats, compact e leave 22
7.9 Ferramentas 22

08 Console web 23
8.1 Abrindo 23
8.2 Visão geral 23
8.3 Nós e anel 24
8.4 Chaves 24
8.5 Terminal 25
8.6 Tema e celular 25

09 Operando o cluster 26
9.1 O nó lemed 26
9.2 Subindo um cluster 27
9.3 Como serviço do systemd 28
9.4 Crescendo o cluster 28
9.5 Retirando um nó 28
9.6 Falhas e recuperação 29
9.7 Compactação 29
9.8 Backup 29
9.9 Monitoramento 29

10 Segurança 31
10.1 Token 31
10.2 Certificados para TLS e mTLS 31
10.3 Console 31
10.4 Boas práticas 32

11 Desempenho e escala 33
11.1 Medindo 33
11.2 O que ajustar 33
11.3 Limites 33

Copyright (©) 2026 Erik Pimenta

12 Testes, build e CI 35
12.1 Comandos 35
12.2 O que os testes cobrem 35
12.3 Integração contínua 35

13 Solução de problemas 36
13.1 Conexão 36
13.2 Dados 36
13.3 Terminal 36

14 Licenças 38
14.1 Componentes de terceiros 38
14.2 Licença do Leme 38

Copyright (©) 2026 Erik Pimenta

01
Apresentação

O Leme é uma linha de comando para conversar com um cluster distribuído de armazenamento chave-valor. Ele tem dois jeitos de trabalhar: um terminal interativo, em que você se conecta uma vez e executa vários comandos, e um modo direto, em que cada chamada executa um comando e sai, pronto para scripts e automação.

1.1 O que vem no projeto

Parte	O que é
`leme`	A CLI. Modo interativo, modo direto, console web, cluster local de testes, carga sintética e perfis de conexão
`lemed`	O nó do cluster. Guarda as chaves em disco, replica as escritas e atende a CLI
Console web	Painel no navegador com visão geral, nós, anel, navegador de chaves e um terminal. Vem dentro do binário `leme`
Docker Compose	Três nós e o console, com volumes e tokens gerados na primeira execução
Scripts `iniciar`	`iniciar.cmd` no Windows e `iniciar.sh` no Linux e no macOS: sobem tudo com um clique
Testes	Testes de unidade, testes de integração com clusters reais em memória e um teste de ponta a ponta com processos separados
Manuais	Este manual e o PDF de arquitetura do banco de dados

1.2 Tecnologias

Camada	Tecnologia
Linguagem	Go 1.24, binários estáticos sem CGO
Linha de comando	Cobra e pflag
Rede	TCP com protocolo binário próprio, multiplexado, com TLS 1.2+ opcional e mTLS
Terminal	Editor de linha próprio, em modo cru, para Linux, macOS, BSD e Windows
Armazenamento	Log de registros com CRC32C, índice em memória e compactação

Copyright (©) 2026 Erik Pimenta

Camada	Tecnologia
Console	HTML, CSS e JavaScript sem frameworks, embutidos com `go:embed`
Tipografia	Instrument Sans, Bricolage Grotesque e IBM Plex Mono
Observabilidade	`/saude` em JSON e `/metricas` no formato do Prometheus
Integração contínua	GitHub Actions

1.3 Estrutura de pastas

```
cmd/leme            CLI
cmd/lemed           nó do cluster
internal/cli        comandos Cobra, flags globais, perfis, dev, bench e console
internal/comandos   registro único dos comandos (CLI, REPL e terminal web)
internal/repl       modo interativo
internal/terminal   editor de linha e modo cru por sistema operacional
internal/saida      tabela, JSON e texto
internal/config     perfis de conexão
internal/cliente    cliente do cluster
internal/no         nó: coordenação, réplicas, gossip, dicas e métricas
internal/armazenamento  log, índice, recuperação e compactação
internal/anel       hash consistente
internal/protocolo  quadros e mensagens
internal/transporte conexões, pool e TLS
internal/console    servidor e páginas do console web
scripts             teste de ponta a ponta e criação do .env no Windows
docs                manuais e imagens
```

1.4 Como ler este manual

Os capítulos 2 a 4 levam você do zero a um cluster funcionando. Os capítulos 5 a 8 explicam o uso do dia a dia: o modo interativo, os scripts, a referência de cada comando e o console web. Os capítulos 9 a 11 são para quem opera o cluster em produção. Comandos aparecem em `fonte mono`; o que muda de instalação para instalação aparece entre sinais de menor e maior, como `<chave>`.

Convenção dos exemplos Os exemplos usam o binário como `leme`. No Windows, sem o binário no PATH, use `.\bin\leme.exe`.

Copyright (©) 2026 Erik Pimenta

02
Instalação

O Leme roda de três formas: com Docker, que sobe três nós e o console; direto com o Go, usando o cluster local embutido; ou com os binários, em máquinas separadas.

2.1 Requisitos

Para rodar com	Você precisa	Observação
Docker	Docker com Docker Compose v2	Sobe 3 nós `lemed` e o console, cada nó com o seu volume
Go	Go 1.24 ou mais novo	Compila os dois binários em poucos segundos
Binários	Nada além do binário	Linux, Windows e macOS, em amd64 e arm64
Navegador	Chrome, Edge, Firefox ou Safari atuais	Só para o console web

2.2 Com Docker

1. Baixe o projeto pelo botão **Code › Download ZIP** do GitHub e descompacte, ou clone com `git clone https://github.com/ravelroot/Distributed-Key-Value-CLI.git`.
2. No Windows, dê dois cliques em `iniciar.cmd`. No Linux ou no macOS, abra um terminal na pasta e rode `./iniciar.sh`.
3. Na primeira vez, o script cria o arquivo `.env` a partir do `.env.example`, com um token aleatório para o cluster (`LEME_TOKEN`) e outro para o console (`LEME_TOKEN_CONSOLE`).
4. O script roda `docker compose up -d --build`, espera os nós ficarem saudáveis e abre o navegador no endereço do console, já com o token.

Comandos úteis depois da primeira vez:

Comando	O que faz
`docker compose up -d --build`	Sobe ou atualiza depois de mudar o código
`docker compose ps`	Mostra os contêineres e se estão saudáveis
`docker compose logs -f no1`	Acompanha os logs de um nó
`docker compose exec console leme --nos no1:7400`	Abre o modo interativo dentro da rede do cluster
`docker compose stop no3`	Derruba um nó para ver o cluster degradado

Copyright (©) 2026 Erik Pimenta

Comando	O que faz
`docker compose down`	Para tudo, mantendo os dados
`docker compose down -v`	Para e apaga os volumes

Por que usar a CLI dentro do contêiner Os nós se anunciam pelos nomes da rede do Docker (`no1:7400`). A CLI do seu computador também funciona com `--nos 127.0.0.1:7401,127.0.0.1:7402,127.0.0.1:7403` : ela percebe que os nomes internos não respondem e passa a coordenar pelas portas publicadas.

2.3 Sem Docker

Com o Go instalado:

```
git clone https://github.com/ravelroot/Distributed-Key-Value-CLI.git
cd Distributed-Key-Value-CLI
go build -o bin/ ./cmd/...
./bin/leme dev --console 127.0.0.1:8080
```

O `leme dev` sobe três nós no mesmo processo, nas portas 7401, 7402 e 7403, com os dados na pasta de configuração do usuário (`~/.config/leme/dev` no Linux, `%AppData%\leme\dev` no Windows). Os dados sobrevivem a reinícios; `--limpar` começa do zero. Em outro terminal:

```
./bin/leme --nos 127.0.0.1:7401
```

2.4 Binários prontos

Cada execução do CI publica os binários de Linux (amd64 e arm64), Windows (amd64) e macOS (arm64) como artefatos. Para compilar você mesmo para outra plataforma:

```
GOOS=windows GOARCH=amd64 CGO_ENABLED=0 go build -o dist/leme.exe ./cmd/leme
GOOS=linux GOARCH=arm64 CGO_ENABLED=0 go build -o dist/lemed ./cmd/lemed
```

O `Makefile` faz o mesmo com a versão e o commit gravados no binário: `make build` . Confira com `leme version` .

2.5 Primeiro acesso

1 Abra o modo interativo: `leme --nos 127.0.0.1:7401` .

Copyright (©) 2026 Erik Pimenta

2. A primeira linha mostra a versão; a segunda, o cluster, quantos nós estão vivos, o número de réplicas e a consistência.
3. Grave uma chave: `put usuario:1 '{"nome":"Ana"}'`.
4. Leia: `get usuario:1`. Para ver versão, TTL e réplicas: `get usuario:1 --detalhes`.
5. Veja o cluster: `cluster status` e `cluster nodes`.
6. Digite `ajuda` para a lista de comandos e `sair` (ou `Ctrl+D`) para encerrar.

2.6 Autocompletar do shell

O modo interativo já completa com `Tab`. Para o shell do sistema completar comandos e opções do modo direto:

Shell	Comando
Bash	`leme completion bash > /etc/bash_completion.d/leme`
Zsh	`leme completion zsh > "${fpath[1]}/_leme"`
Fish	`leme completion fish > ~/.config/fish/completions/leme.fish`
PowerShell	`leme completion powershell | Out-String | Invoke-Expression` (adicione ao `$PROFILE`)

Copyright (©) 2026 Erik Pimenta

03
Conceitos

Este capítulo explica as ideias que aparecem nas saídas dos comandos. O PDF de arquitetura do banco de dados aprofunda cada uma.

3.1 Cluster e nós

Um **cluster** é um grupo de processos `lemed`, os **nós**, que juntos guardam as chaves. Todo nó tem um identificador (`no-1`), um endereço (`10.0.0.11:7400`) e um estado: `vivo`, `suspeito` (sem resposta há 3 segundos), `morto` (há 10 segundos) ou `saiu` (retirado com `node leave`). Os nós descobrem uns aos outros por **gossip**: a cada segundo, cada nó troca a lista de membros com os outros.

3.2 Anel e réplicas

As chaves são distribuídas por **hash consistente**. Cada nó ocupa 128 posições, os **nós virtuais**, num anel de 2^{64} posições. Uma chave cai numa posição e pertence aos `N` nós seguintes no sentido horário, as **réplicas**. A primeira é a primária. `leme locate <chave>` mostra quem são; `leme cluster ring` mostra a fatia do anel de cada nó.

3.3 Consistência

Cada operação escolhe quantas réplicas precisam responder:

Consistência	Escrita confirma com	Leitura responde com	Quando usar
`um`	1 réplica	1 réplica	Menor latência, aceita ler um valor um pouco atrasado
`quorum` (padrão)	maioria (2 de 3)	maioria	Equilíbrio: tolera um nó fora e sempre lê a última escrita confirmada
`todos`	todas	todas	Máxima garantia; falha se qualquer réplica estiver fora

Com `quorum` nas leituras e nas escritas, as duas maiorias sempre se cruzam em pelo menos uma réplica, então toda leitura enxerga a última escrita confirmada.

3.4 Versões

Cada escrita recebe uma **versão** de 64 bits gerada pelo nó coordenador: os 48 bits de cima são o horário em milissegundos e os 16 de baixo, um contador. Quando duas réplicas discordam, vence a versão maior. `get --`

Copyright (©) 2026 Erik Pimenta

`detalhes` mostra a versão e o horário da gravação; `put --se-versao` usa a versão para escritas condicionais.

3.5 TTL e lápides

Uma chave gravada com `--ttl 30m` expira sozinha em 30 minutos. Apagar uma chave grava uma **lápide**, uma marca versionada de que ela foi apagada, para que as réplicas atrasadas não a ressuscitem. Lápides e chaves expiradas saem do disco na **compactação**.

3.6 Dicas e reparo

Se uma réplica está fora do ar durante uma escrita, o coordenador guarda uma **dica** e entrega a escrita quando ela voltar. Se uma leitura percebe uma réplica atrasada, envia a versão nova para ela: é o **reparo de leitura**. `cluster status` mostra quantas dicas estão pendentes e quantos reparos foram feitos.

Copyright (©) 2026 Erik Pimenta

04
Conectando ao cluster

4.1 Informando os nós

A CLI precisa de pelo menos um nó; ela descobre os outros sozinha. Informe vários para não depender de um só:

```
leme --nos 10.0.0.11:7400,10.0.0.12:7400 cluster status
```

Sem `--nos`, a CLI usa a variável `LEME_NOS`, depois o perfil atual e, por último, `127.0.0.1:7400`.

4.2 Perfis de conexão

Um perfil guarda nós, token, TLS, consistência, tempo limite e formato de saída. O primeiro perfil criado vira o padrão.

```
leme config set local --nos 127.0.0.1:7401,127.0.0.1:7402
leme config set producao --nos 10.0.0.11:7400,10.0.0.12:7400 \
    --token-arquivo ~/.leme-token --tls-ca ca.pem --consistencia quorum --tempo-limite 3s
leme config list
leme config use producao
leme --perfil local get usuario:1
leme config view
```

Comando	O que faz
`config set <perfil>`	Cria ou altera; só as opções informadas mudam
`config use <perfil>`	Define o perfil padrão
`config list`	Lista, com `*` no padrão
`config view`	Mostra a conexão que será usada, sem revelar o token
`config delete <perfil>`	Apaga

O arquivo fica em `~/.config/leme/config.json` (Linux), `~/Library/Application Support/leme/config.json` (macOS) ou `%AppData%\leme\config.json` (Windows), com permissão de leitura só para o dono. `LEME_CONFIG` ou `--config` usam outro arquivo.

Copyright (©) 2026 Erik Pimenta

4.3 Ordem de prioridade

Para cada opção, vale a primeira que existir:

1. A flag na linha de comando (`--nos` , `--token` , `-o` , `-c`).
2. A variável de ambiente (`LEME_NOS` , `LEME_TOKEN` , `LEME_SAIDA` , `LEME_CONSISTENCIA` , `LEME_PERFIL`).
3. O perfil escolhido com `--perfil` , ou o perfil padrão.
4. O valor de fábrica.

4.4 Token

Se o cluster exige token, a CLI envia o token logo depois de conectar. Prefira `--token-arquivo` ou a variável `LEME_TOKEN` a `--token` , que fica no histórico do shell. Um token errado encerra o comando com o código 7.

4.5 TLS

Com nós em TLS, informe a CA que assinou os certificados deles:

```
leme --nos no1.interno:7400 --tls-ca ca.pem cluster status
```

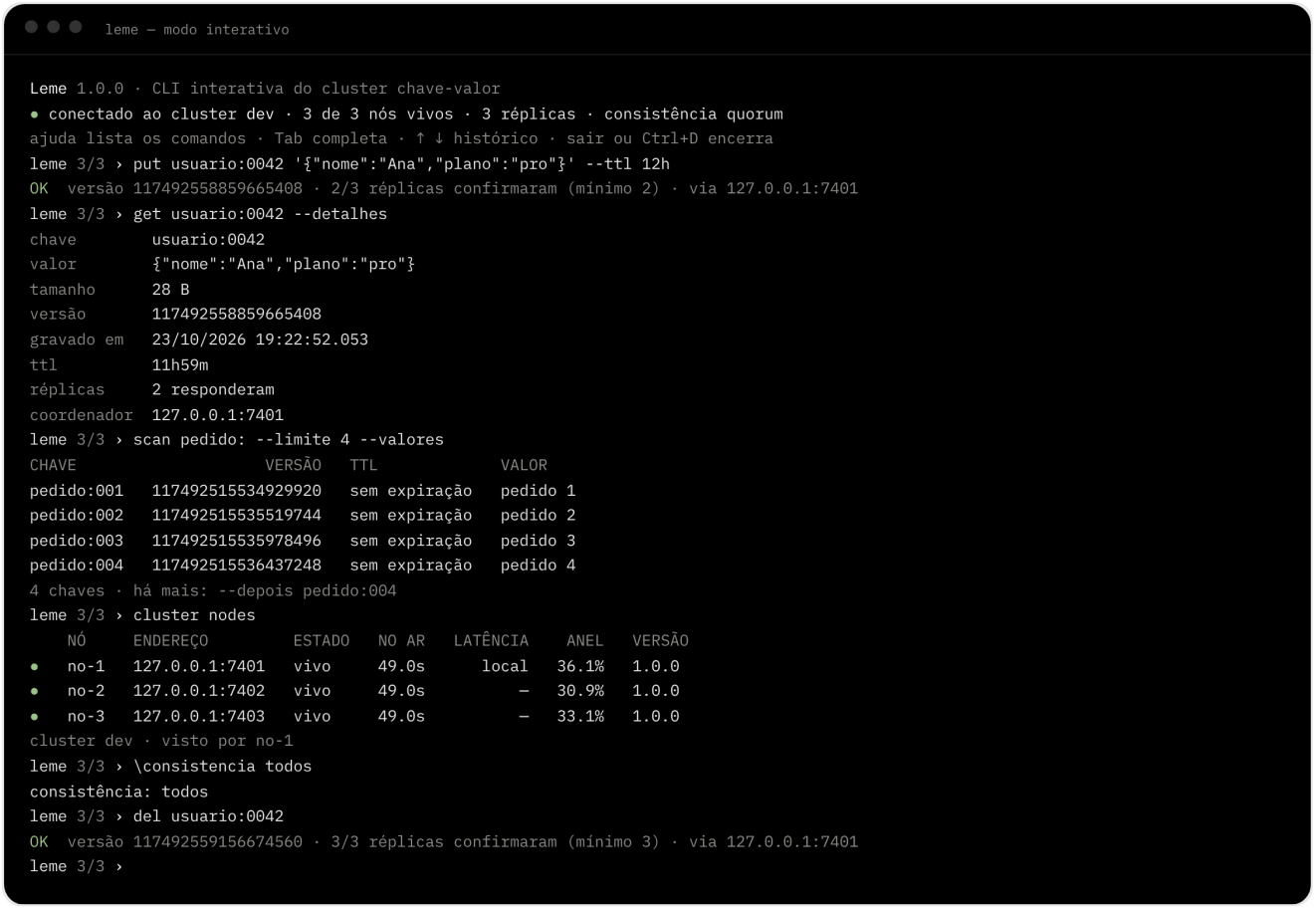
Se os nós exigem certificado do cliente (mTLS), acrescente `--tls-cert cliente.pem --tls-chave cliente.key` . `--tls-inseguro` aceita qualquer certificado e serve só para testes. O capítulo 10 mostra como gerar os certificados.

Copyright (©) 2026 Erik Pimenta

05
Modo interativo

5.1 Abrindo

`leme` sem comando, ou `leme shell`, abre o modo interativo quando a entrada é um terminal. A conexão é aberta uma vez e reaproveitada por todos os comandos.



O prompt `leme 3/3 ›` mostra quantos nós estão vivos do total conhecido. A sessão começa com a consistência e o formato do perfil.

5.2 Editando a linha

Tecla	O que faz
`Tab`	Completa comando, subcomando, opção, valor de opção ou chave já usada; com várias opções, mostra a lista

Copyright (©) 2026 Erik Pimenta

Tecla	O que faz
`↑` `↓` ou `Ctrl+P` `Ctrl+N`	Navega no histórico
`←` `→` ou `Ctrl+B` `Ctrl+F`	Move o cursor
`Ctrl+←` `Ctrl+→` ou `Alt+B` `Alt+F`	Pula palavras
`Home` `End` ou `Ctrl+A` `Ctrl+E`	Início e fim da linha
`Ctrl+W`	Apaga a palavra anterior
`Ctrl+U` · `Ctrl+K`	Apaga até o início · até o fim
`Ctrl+L`	Limpa a tela
`Ctrl+C`	Cancela a linha, ou o comando em andamento, sem sair
`Ctrl+D`	Sai, com a linha vazia

O histórico guarda até 1.000 comandos, sem repetir o anterior, e fica no arquivo `historico` da pasta de configuração.

5.3 Valores com espaços e caracteres especiais

A linha é dividida em palavras como num shell:

Você digita	O valor gravado
`put nota:1 "texto com espaços"`	`texto com espaços`
`put perfil:1 '{"nome": "Ana"}'`	`{"nome": "Ana"}` (aspas simples preservam tudo)
`put k "aspas \"dentro\""`	`aspas "dentro"`
`put k linha\nnova`	duas linhas
`put saldo -5`	`-5` (números negativos não viram opção)

5.4 Comandos de sessão

Comando	O que faz
`ajuda` ou `\?`	Lista os comandos por grupo
`ajuda <comando>`	Opções, apelidos e exemplos do comando

Copyright (©) 2026 Erik Pimenta

Comando	O que faz
`\saida tabela|json|texto`	Muda o formato das próximas respostas
`\consistencia um|quorum|todos`	Muda a consistência das próximas operações
`\tempo`	Mostra o tempo de cada comando
`\conectar host:porta[,host:porta]`	Troca de cluster sem sair
`\historico [n]`	Lista os últimos comandos
`\nos`	Atalho para `cluster nodes`
`limpar`	Limpa a tela
`sair`, `exit` ou `\q`	Encerra

Linhas que começam com `#` são comentários.

5.5 Confirmações

Comandos que mudam o cluster, como `node leave`, pedem confirmação no modo interativo. Responda `s` para seguir. Nos scripts, a confirmação é a opção `--confirmar`.

Copyright (©) 2026 Erik Pimenta

06
Scripts e automação

6.1 Modo direto

Qualquer comando pode ser executado direto. A CLI conecta, executa, imprime e sai:

```
leme put config:tema escuro
TEMA=$(leme get config:tema)
```

A saída de `get` é sempre o valor puro, sem enfeites, para caber em variáveis e pipes. Mensagens de erro vão para a saída de erro, nunca para a saída padrão.

6.2 Formatos de saída

Formato	Quando usar	Exemplo
`tabela` (padrão)	Leitura humana; cores só quando a saída é um terminal	`leme cluster nodes`
`json`	Automação; objetos indentados, versões como texto para não perder precisão	`leme get k -o json | jq -r .valor`
`texto`	Pipes; uma linha por item, colunas separadas por tabulação	`leme keys pedido: -o texto`

6.3 Códigos de saída

Código	Significado	Exemplo
`0`	Sucesso	
`1`	Erro inesperado	Disco cheio num nó
`2`	Uso incorreto	Faltou a chave, opção desconhecida
`3`	Chave não encontrada	`get` ou `exists` de chave ausente
`4`	Cluster indisponível	Nenhum nó respondeu, tempo esgotado
`5`	Quórum não atingido	`-c todos` com uma réplica fora
`6`	Conflito de versão	`--se-versao` ou `--se-ausente` falharam

Copyright (©) 2026 Erik Pimenta

Código	Significado	Exemplo
`7`	Token recusado	

```
if leme exists usuario:42; then echo "já existe"; fi

leme put estoque:7 41 --se-versao "$V"
case $? in
  0) echo "gravado" ;;
  6) echo "outra pessoa alterou antes, leia de novo" ;;
  *) echo "falhou" ;;
esac
```

6.4 Entrada padrão e arquivos

```
cat relatorio.json | leme put relatorio:2026-10 -
leme put foto:1 --arquivo foto.png
leme get foto:1 > copia.png
```

Valores binários são gravados e lidos byte a byte. Na tabela, aparecem como `<N bytes binários>`; no JSON, em `valor_base64`.

6.5 Arquivos de comandos

Um arquivo com um comando por linha roda com `leme shell --arquivo` ou por pipe:

```
# carga.leme
put usuario:1 '{"nome":"Ana"}'
put usuario:2 '{"nome":"Bruno"}'
\saida json
count usuario:
```

```
leme shell --arquivo carga.leme
leme shell --arquivo carga.leme --parar-no-erro
printf 'put a 1\nget a\n' | leme
```

Sem `--parar-no-erro`, todas as linhas rodam e o código de saída é o da primeira falha. Com ele, o script para na primeira falha.

Copyright (©) 2026 Erik Pimenta

6.6 PowerShell

```
$env:LEME_NOS = "127.0.0.1:7401"
leme put sessao:1 ativa --ttl 15m
$usuario = leme get usuario:1 -o json | ConvertFrom-Json
if ($LASTEXITCODE -eq 3) { Write-Host "não existe" }
leme keys pedido: -o texto | ForEach-Object { leme del $_ }
```

Copyright (©) 2026 Erik Pimenta

07

Referência de comandos

Todos os comandos aceitam as opções globais `--nos`, `--perfil`, `--token`, `--token-arquivo`, `--tls-ca`, `--tls-cert`, `--tls-chave`, `--tls-inseguro`, `--tempo-limite`, `-o` / `--saida`, `--sem-cor` e `--config`. Os comandos de dados aceitam `-c` / `--consistencia`.

7.1 get

`leme get <chave>` · apelidos `ler`, `obter`. Lê o valor mais recente entre as réplicas consultadas.

Opção	O que faz
`-d`, `--detalhes`	Mostra tamanho, versão, horário da gravação, TTL, réplicas que responderam e coordenador
`-c`, `--consistencia`	`um`, `quorum` ou `todos`

```
leme get usuario:42
leme get usuario:42 --detalhes -c todos
leme get config:tema -o json
```

7.2 put

`leme put <chave> <valor | ->` · apelidos `set`, `gravar`. Grava um valor.

Opção	O que faz
`-t`, `--ttl`	Expira depois da duração: `90` (segundos), `30s`, `15m`, `2h`
`--se-versao <v>`	Só grava se a versão atual for `v` (código 6 se não for)
`-n`, `--se-ausente`	Só grava se a chave não existir
`-f`, `--arquivo`	Lê o valor de um arquivo
`-c`, `--consistencia`	Réplicas que precisam confirmar

A resposta mostra a versão gravada, quantas réplicas confirmaram até a resposta, o mínimo exigido e o nó coordenador. Limites: chave até 64 KiB, valor até 32 MiB.

Copyright (©) 2026 Erik Pimenta

7.3 del

`leme del <chave>...` · apelidos `delete`, `apagar`, `rm`. Apaga uma ou várias chaves. `--se-versao` funciona com uma chave por vez. Apagar uma chave que não existe também grava a lápide e termina com sucesso.

7.4 exists, ttl e expire

Comando	O que faz	Saída em texto
`exists <chave>`	Código 0 se existe, 3 se não	`1` ou `0`
`ttl <chave>`	Tempo até expirar	milissegundos, ou `-1` sem expiração
`expire <chave> <duração>`	Novo TTL a partir de agora; `0` remove a expiração	`OK`

7.5 scan, keys e count

`leme scan [prefixo]` · apelidos `varrer`, `listar`. Lista as chaves em ordem lexicográfica, consultando todos os nós vivos.

Opção	O que faz
`-l`, `--limite`	Chaves por página, de 1 a 10.000 (padrão 50)
`--depois <chave>`	Começa depois desta chave: a próxima página
`-v`, `--valores`	Inclui os valores
`-a`, `--todas`	Percorre todas as páginas

O rodapé indica se há mais páginas e a chave para continuar. `keys [prefixo]` lista só os nomes (até `--limite`, padrão 1.000). `count [prefixo]` conta percorrendo as páginas.

7.6 locate e ping

`locate <chave>` (apelido `localizar`) mostra a réplica primária e as demais, com endereço e estado. `ping [nó]` mede a latência até cada nó, ou até um só, e termina com código 4 se nenhum responder.

Copyright (©) 2026 Erik Pimenta

7.7 cluster status, nodes e ring

Comando	Mostra
`cluster status`	Saúde (saudável, degradado ou sem quórum), nós por estado, replicação, chaves aproximadas, disco, operações, latência p99 de leitura e escrita, dicas e reparos
`cluster nodes` (`nos` , `membros`)	Cada membro com estado, endereço, tempo no ar, latência medida pelo gossip, fatia do anel e versão
`cluster ring` (`anel`)	A fatia do espaço de chaves de cada nó, com barra proporcional

7.8 node stats, compact e leave

Comando	O que faz
`node stats <nó>`	Versão, tempo no ar, TLS e token, conexões, memória, chaves, lápides, disco, fragmentação, fsync, compactações, dicas, reparos e a latência de cada operação (média, p50, p95 e p99)
`node compact <nó | todos>`	Compacta os segmentos e mostra o tamanho antes e depois, os registros removidos e o tempo
`node leave <nó> --confirmar`	Retira o nó do cluster: ele anuncia a saída, o anel é recalculado e as chaves vão para as novas réplicas

O `<nó>` pode ser o identificador (`no-2`) ou o endereço (`10.0.0.12:7400`).

7.9 Ferramentas

Comando	O que faz
`shell`	Modo interativo; `--arquivo` e `--parar-no-erro` para scripts
`console`	Console web; `--endereco` e `--token-console`
`dev`	Cluster local: `--quantidade` , `--porta` , `--replicas` , `--dados` , `--console` , `--logs` , `--limpar`
`bench`	Carga sintética: `--operacoes` , `--concorrencia` , `--tamanho` , `--chaves` , `--leituras` , `--prefixo`
`config`	Perfis de conexão
`version`	Versão, Go, sistema e commit
`completion`	Autocompletar do shell

Copyright (©) 2026 Erik Pimenta

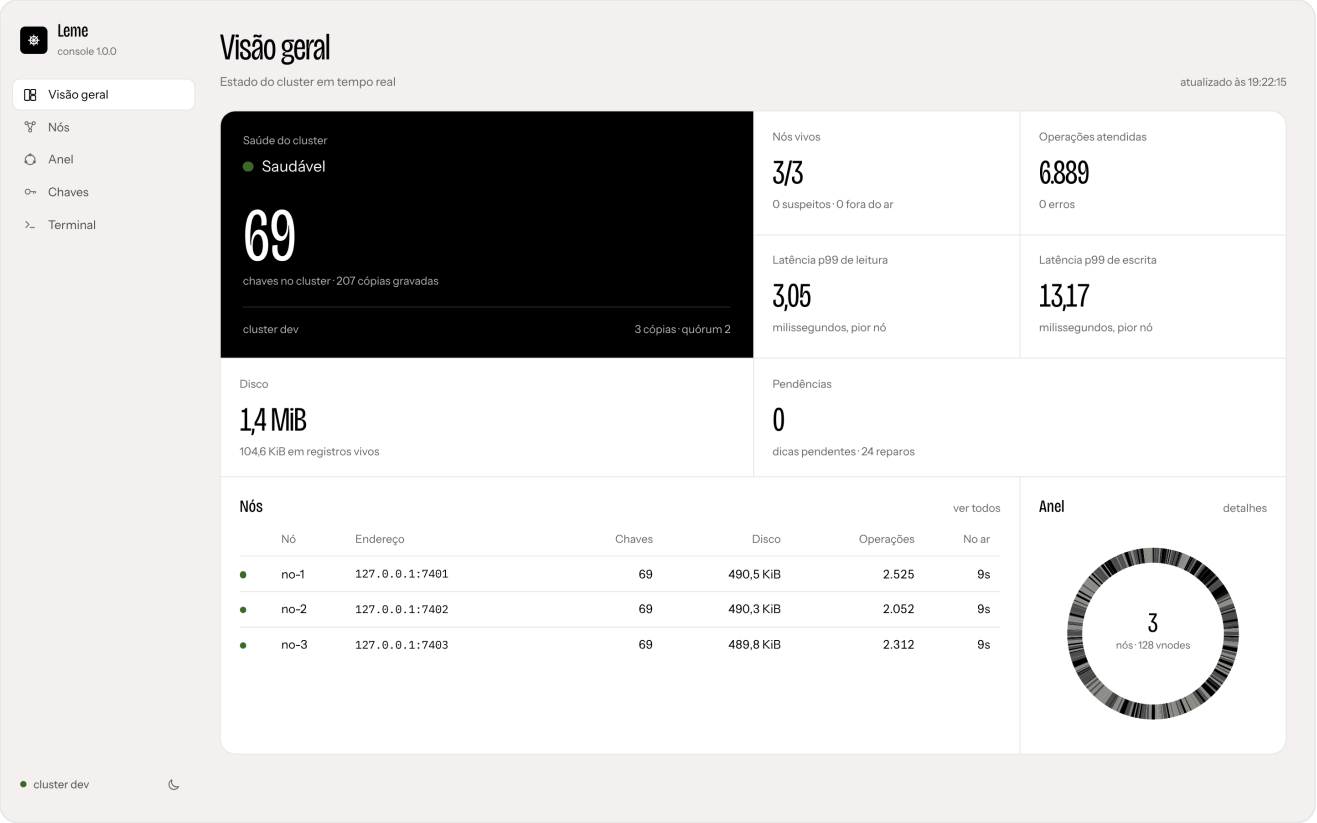
08
Console web

8.1 Abrindo

```
leme console                                          # http://127.0.0.1:8080/
leme --perfil producao console --endereco 127.0.0.1:9090
leme console --endereco 0.0.0.0:8080 --token-console "$(openssl rand -hex 16)"
```

O console usa a mesma conexão da CLI. Por segurança, escuta só em `127.0.0.1`; para expor na rede é obrigatório definir `--token-console` (ou `LEME_TOKEN_CONSOLE`). O endereço mostrado no terminal já leva o token; o navegador troca o token por um cookie e remove-o da barra de endereço.

8.2 Visão geral



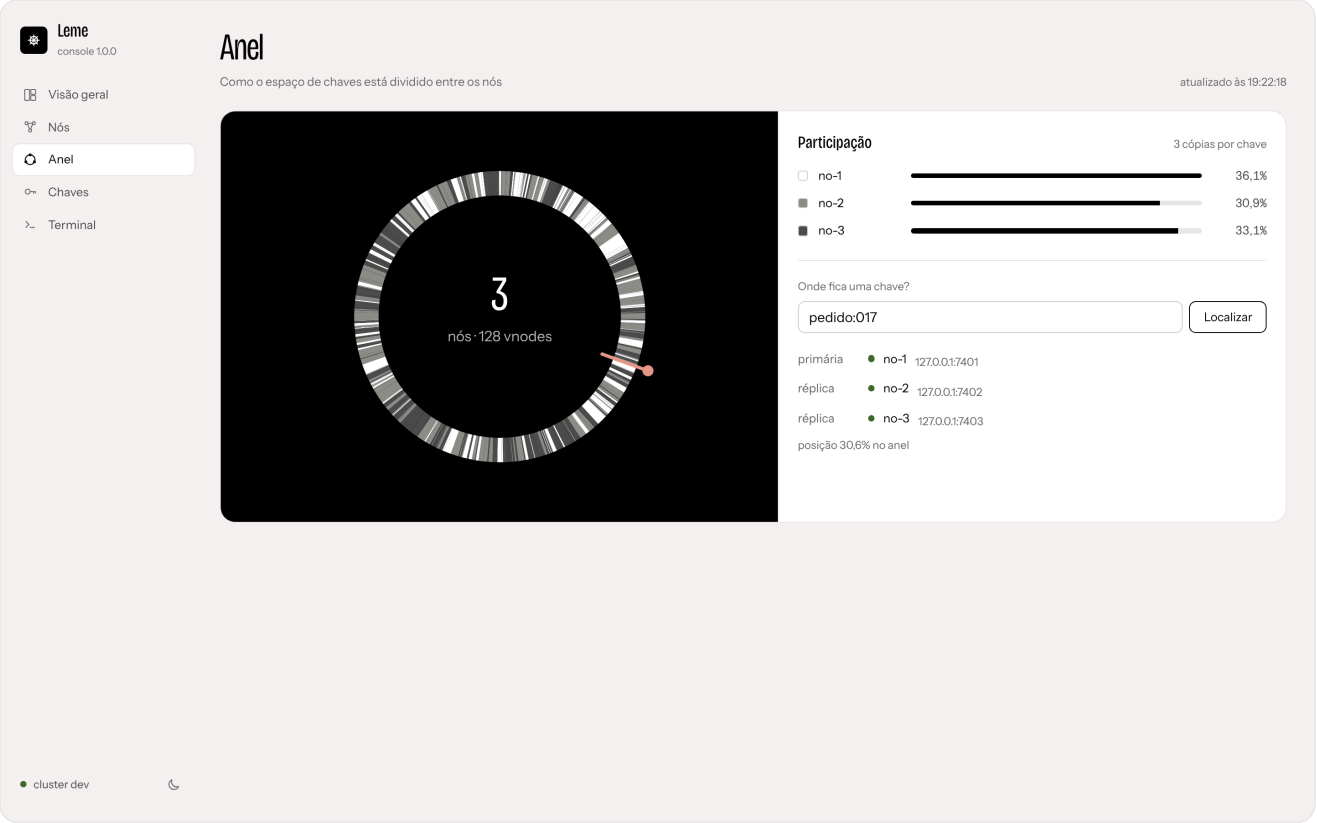
O bloco escuro mostra a saúde do cluster e a quantidade de chaves. Ao lado ficam os nós vivos, as operações atendidas, a latência p99 de leitura e de escrita (do pior nó), o disco e as pendências. Embaixo, a tabela dos nós e

Copyright (©) 2026 Erik Pimenta

o anel. A página se atualiza a cada 3 segundos.

8.3 Nós e anel

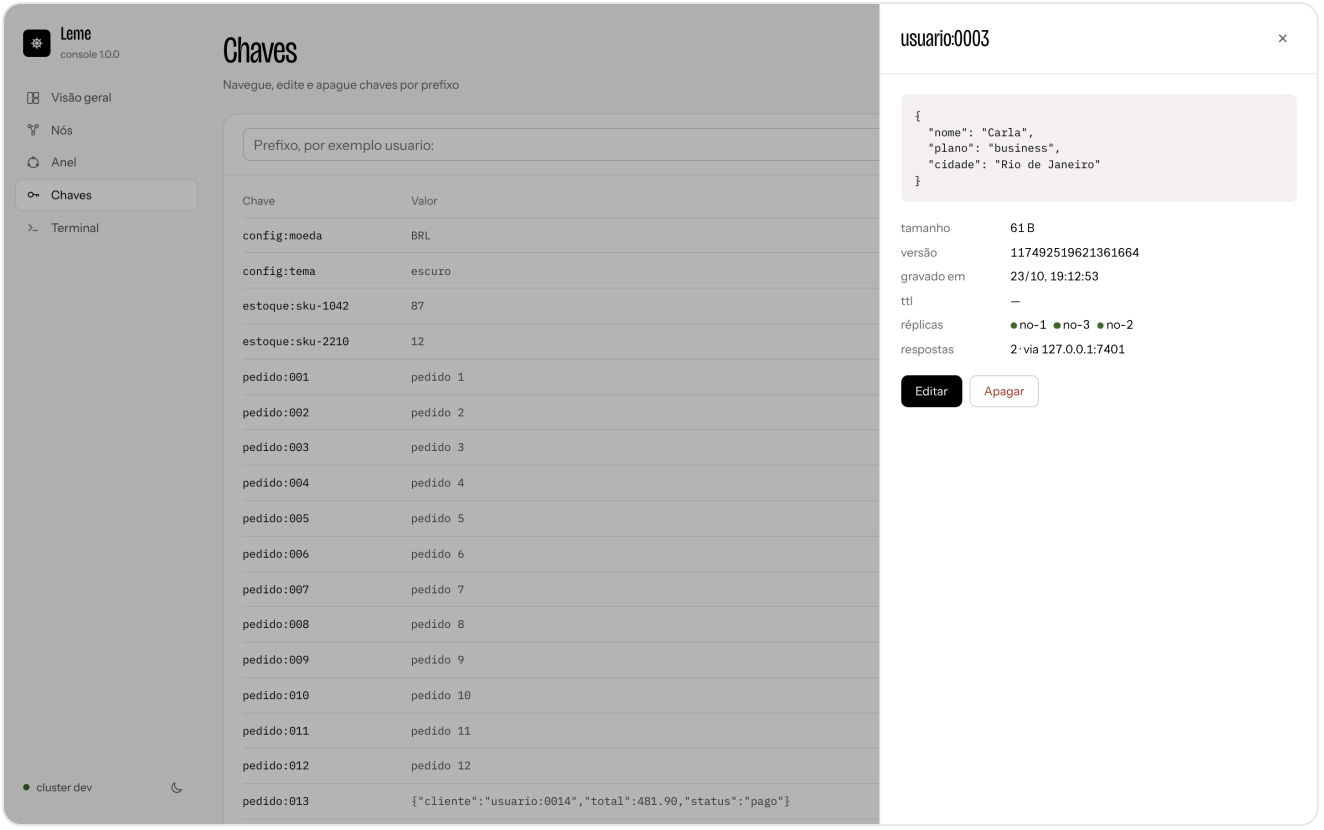
A página **Nós** mostra, para cada membro, estado, endereço, chaves, lápides, disco, fragmentação, operações, p99 de leitura, fatia do anel e tempo no ar. A página **Anel** desenha as faixas de cada nó e responde onde fica uma chave: digite a chave, clique em **Localizar** e veja a posição no anel, a primária e as réplicas.



8.4 Chaves

Busque por prefixo e carregue mais páginas com **Carregar mais**. Clique numa linha para abrir a gaveta com o valor (JSON aparece formatado), tamanho, versão, horário, TTL, réplicas e o nó que respondeu. Na gaveta, **Editar** grava com a opção de só gravar se ninguém alterou desde a versão aberta, e **Apagar** pede confirmação. **Nova chave** abre um formulário com chave, valor e TTL.

Copyright (©) 2026 Erik Pimenta



8.5 Terminal

O terminal web executa os mesmos comandos da CLI, com histórico em `↑` e `↓` e `Ctrl+L` para limpar. Comandos perigosos exigem `--confirmar`.

8.6 Tema e celular

O botão da lua alterna entre claro e escuro, e a escolha fica salva no navegador. No celular, o menu vira uma gaveta e os blocos ficam em uma coluna.

Copyright (©) 2026 Erik Pimenta

09

Operando o cluster

9.1 O nó lemed

Flag	Variável	Padrão	O que faz
`--id`	`LEME_ID`	`host-porta`	Identificador único e estável do nó
`--cluster`	`LEME_CLUSTER`	`leme`	Nome do cluster
`--endereco`	`LEME_ENDERECO`	`127.0.0.1:7400`	Onde escuta
`--anunciar`	`LEME_ANUNCIAR`	o endereço	Endereço que os outros usam para chegar até ele
`--entrar`	`LEME_ENTRAR`	vazio	Nós existentes para entrar no cluster
`--dados`	`LEME_DADOS`	`dados`	Pasta dos dados; cada nó usa a subpasta com o seu id
`--replicas`	`LEME_REPLICAS`	`3`	Cópias de cada chave
`--vnodes`	`LEME_VNODES`	`128`	Nós virtuais por nó
`--token`, `--token-arquivo`	`LEME_TOKEN`, `LEME_TOKEN_ARQUIVO`	vazio	Token de clientes e pares
`--tls-cert`, `--tls-chave`, `--tls-ca`	`LEME_TLS_*`	vazio	TLS e mTLS
`--metricas`	`LEME_METRICAS`	vazio	HTTP de `/saude` e `/metricas`
`--sincronizar`	`LEME_SINCRONIZAR`	`intervalo`	`sempre`, `intervalo` ou `nunca`
`--segmento-mb`	`LEME_SEGMENTO_MB`	`64`	Tamanho de cada segmento de log
`--graca-lapide`	—	`24h`	Tempo mínimo de uma lápide no disco
`--gossip`	—	`1s`	Intervalo do gossip

Copyright (©) 2026 Erik Pimenta

Flag	Variável	Padrão	O que faz
`--tempo-operacao`	—	`3s`	Limite de uma operação entre nós
`--max-conexoes`	`LEME_MAX_CONEXOES`	`4096`	Conexões simultâneas
`--log`, `--log-formato`	`LEME_LOG`, `LEME_LOG_FORMATO`	`info`, `texto`	Logs; `json` para coletores

Mesmos valores em todos os nós `--cluster`, `--replicas`, `--vnodes` e o token precisam ser iguais em todos os nós. Nós de clusters com nomes diferentes recusam a conversa.

9.2 Subindo um cluster

```
lemed --id no-1 --endereco 0.0.0.0:7400 --anunciar 10.0.0.11:7400 --dados /var/lib/leme \
      --token-arquivo /etc/leme/token --metricas 0.0.0.0:9400
lemed --id no-2 --endereco 0.0.0.0:7400 --anunciar 10.0.0.12:7400 --dados /var/lib/leme \
      --token-arquivo /etc/leme/token --metricas 0.0.0.0:9400 --entrar 10.0.0.11:7400
lemed --id no-3 ... --entrar 10.0.0.11:7400,10.0.0.12:7400
```

Use pelo menos três nós para tolerar a queda de um com quórum. Confira com `leme --nos 10.0.0.11:7400 cluster nodes`.

Copyright (©) 2026 Erik Pimenta

9.3 Como serviço do systemd

```
[Unit]
Description=Leme - no do cluster
After=network-online.target
Wants=network-online.target

[Service]
User=leme
Environment=LEME_ID=no-1
Environment=LEME_ENDERECO=0.0.0.0:7400
Environment=LEME_ANUNCIAR=10.0.0.11:7400
Environment=LEME_DADOS=/var/lib/leme
Environment=LEME_ENTRAR=10.0.0.12:7400,10.0.0.13:7400
Environment=LEME_TOKEN_ARQUIVO=/etc/leme/token
Environment=LEME_METRICAS=0.0.0.0:9400
ExecStart=/usr/local/bin/lemed
Restart=on-failure
LimitNOFILE=65536

[Install]
WantedBy=multi-user.target
```

Ao receber `SIGTERM`, o nó para de aceitar conexões, espera as operações em andamento por até 15 segundos, sincroniza o log e sai.

9.4 Crescendo o cluster

Suba o novo nó com `--entrar` apontando para os existentes. Em segundos ele aparece em `cluster nodes`, o anel é recalculado e cada nó envia ao novo as chaves que passaram a ser dele. Com nós virtuais, um quarto nó recebe perto de 25% das chaves, tiradas um pouco de cada um dos outros.

9.5 Retirando um nó

```
leme node leave no-3 --confirmar
leme cluster nodes                # no-3 aparece como saiu
```

O nó transfere todas as suas chaves para as novas réplicas antes de sair do anel. Depois disso o processo pode ser desligado. Para devolver um nó retirado, apague a pasta de dados dele e suba de novo com `--entrar`.

Copyright (©) 2026 Erik Pimenta

9.6 Falhas e recuperação

Situação	O que acontece	O que fazer
Um nó cai	Vira suspeito em 3s e morto em 10s; com quórum, leituras e escritas seguem; escritas para ele viram dicas	Nada; ao voltar, recebe as dicas e o reparo de leitura completa o resto
Dois de três nós caem	Operações com `quorum` falham com código 5; `-c um` ainda funciona	Suba os nós; `cluster status` mostra "sem quórum"
Queda de energia	Na volta, o nó relê o log, valida o CRC de cada registro e trunca uma escrita incompleta no fim	Nada; veja "registros descartados" em `node stats`
Disco perdido num nó	O nó volta vazio	Retire-o com `node leave` e suba-o de novo, vazio, com outro id e `--entrar`: os outros nós enviam a ele as chaves que passam a ser dele

9.7 Compactação

As escritas só acrescentam registros, então versões antigas, lápides e chaves expiradas ocupam espaço até a compactação. Rode quando a fragmentação de `node stats` passar de uns 50%:

```
leme node compact todos
```

A compactação reescreve os segmentos fechados sem bloquear leituras e escritas e só remove lápides com mais de `--graca-lapide` (24h).

9.8 Backup

Os segmentos são arquivos só de acréscimo. Para um backup consistente de um nó: rode `node compact` (que fecha o segmento ativo), copie a pasta `dados/<id>` e guarde. Como cada chave tem três réplicas, um backup por nó em horários diferentes já protege contra a perda simultânea de todos.

9.9 Monitoramento

Com `--metricas`, cada nó expõe `GET /saude` (200 com `{"status":"ok"}`, 503 durante a saída), `GET /v1/estatisticas`, `GET /v1/topologia` e `GET /metricas`:

Métrica	Tipo	O que mede
`leme_operacoes_total{op}`	contador	Operações atendidas por tipo
`leme_erros_total{op}`	contador	Operações com erro

Copyright (©) 2026 Erik Pimenta

Métrica	Tipo	O que mede
`leme_operacao_segundos{op}`	histograma	Latência
`leme_chaves`, `leme_lapides`	medidor	Registros no nó
`leme_bytes_disco`, `leme_bytes_vivos`, `leme_segmentos`	medidor	Disco e fragmentação
`leme_dicas_pendentes`	medidor	Escritas esperando réplicas fora do ar
`leme_reparos_leitura_total`, `leme_compactacoes_total`	contador	Reparos e compactações
`leme_conexoes`, `leme_memoria_bytes`, `leme_tempo_no_ar_segundos`	medidor	Processo

Alertas úteis: `leme_dicas_pendentes` crescendo por mais de 5 minutos, `1 - leme_bytes_vivos/leme_bytes_disco > 0.6`, p99 de `PUT` acima de 50 ms e `/saude` diferente de 200.

Copyright (©) 2026 Erik Pimenta

10
Segurança

10.1 Token

```
openssl rand -hex 32 > /etc/leme/token && chmod 600 /etc/leme/token
lemed ... --token-arquivo /etc/leme/token
export LEME_TOKEN=$(cat /etc/leme/token)
```

O mesmo token vale para clientes e para a conversa entre nós. Sem token, o nó só responde a `PING`. Cada tentativa errada espera 200 ms e encerra a conexão.

10.2 Certificados para TLS e mTLS

```
openssl req -x509 -newkey rsa:3072 -nodes -days 3650 -subj "/CN=leme-ca" -keyout ca.key -out
ca.pem
for no in no1 no2 no3; do
  openssl req -newkey rsa:3072 -nodes -subj "/CN=$no" -keyout $no.key -out $no.csr
  openssl x509 -req -in $no.csr -CA ca.pem -CAkey ca.key -CAcreateserial -days 825 -out $no.pem \
    -extfile <(printf "subjectAltName=DNS:$no,IP:10.0.0.1%s" "${no#no}")
done
openssl req -newkey rsa:3072 -nodes -subj "/CN=cli" -keyout cli.key -out cli.csr
openssl x509 -req -in cli.csr -CA ca.pem -CAkey ca.key -CAcreateserial -days 825 -out cli.pem
```

```
lemed ... --tls-cert no1.pem --tls-chave no1.key --tls-ca ca.pem
leme --nos no1:7400 --tls-ca ca.pem --tls-cert cli.pem --tls-chave cli.key cluster status
```

Com `--tls-ca` no nó, ele exige certificado de clientes e pares assinado pela CA (mTLS). Sem `--tls-ca`, o TLS só cifra o tráfego.

10.3 Console

O console escuta em `127.0.0.1` por padrão. Exposto, exige token, guardado em cookie `HttpOnly` e `SameSite=Strict`. Pedidos que alteram dados precisam do cabeçalho `X-Leme`, o que bloqueia formulários de outros sites, e todas as respostas levam Content-Security-Policy, `nosniff`, `X-Frame-Options: DENY` e `Referrer-Policy: no-referrer`. Atrás de um proxy reverso com HTTPS, mantenha o token.

Copyright (©) 2026 Erik Pimenta

10.4 Boas práticas

- Coloque os nós numa rede privada e libere a porta 7400 só para os outros nós e para quem opera.
- Use TLS sempre que o tráfego passar por redes compartilhadas.
- Não passe o token por `--token` em servidores compartilhados: ele aparece na lista de processos.
- Rode o `lemed` com um usuário próprio, sem privilégios, como faz a imagem Docker.
- Monitore `/saude` e as dicas pendentes.

Copyright (©) 2026 Erik Pimenta

11

Desempenho e escala

11.1 Medindo

```
leme bench --operacoes 50000 --concorrencia 64 --leituras 0.9 --tamanho 512
```

O bench grava as chaves de teste, mede leituras e escritas misturadas e mostra quantidade, p50, p95, p99 e máxima de cada tipo, além das operações por segundo. Em um notebook, com três nós locais e quórum, passa de 10 mil operações por segundo com 16 clientes. Apague as chaves de teste depois com `leme keys bench: -o texto | xargs leme del`.

11.2 O que ajustar

Ajuste	Efeito
`-c um` nas leituras	Menor latência; aceita ler um valor um pouco atrasado
`--sincronizar sempre`	Nenhuma escrita confirmada se perde numa queda de energia; escritas mais lentas
`--sincronizar nunca`	Escritas mais rápidas; o sistema operacional decide quando gravar
`--replicas 5`	Tolera duas quedas com quórum; mais tráfego de rede e disco
`--vnodes 256`	Distribuição mais uniforme com poucos nós; anel um pouco maior na memória
`--segmento-mb`	Segmentos menores compactam mais rápido; maiores geram menos arquivos
Mais nós	Mais capacidade total e menos chaves por nó

11.3 Limites

Item	Limite
Chave	64 KiB
Valor	32 MiB
Mensagem do protocolo	64 MiB
Página de scan	10.000 chaves

Copyright (©) 2026 Erik Pimenta

Item	Limite
Conexões por nó	4.096 (ajustável)
Requisições simultâneas por conexão	256
Dicas guardadas por coordenador	100.000 chaves

O índice de cada nó fica na memória: conte cerca de 100 bytes mais o tamanho da chave por registro. Um milhão de chaves de 30 bytes por nó ocupa perto de 130 MiB.

Copyright (©) 2026 Erik Pimenta

12
Testes, build e CI

12.1 Comandos

Comando	O que faz
`go test -race ./...`	Todos os testes, com o detector de corridas
`bash scripts/ponta-a-ponta.sh bin`	Três processos `lemed` e a CLI, como no CI
`go vet ./...` e `gofmt -l .`	Análise estática e formatação
`make build`	Binários com versão e commit
`make dev`	Compila e sobe o cluster local com o console

12.2 O que os testes cobrem

Área	Testes
Armazenamento	Gravar, ler e reabrir; última escrita vence; lápides; TTL; cauda corrompida; bit trocado; compactação; varredura
Anel	Distribuição equilibrada; réplicas distintas; estabilidade ao crescer; ordem dos membros
Protocolo	Ida e volta; mensagens truncadas; quadros grandes e versão errada
Cluster	Quórum com nó derrubado; consistência `todos`; escrita condicional; TTL; token; dicas entregues ao nó que volta; saída de nó sem perda; varredura sobre lápides
CLI	Comandos diretos, JSON, códigos de saída, entrada padrão, modo não interativo, perfis
Console	Páginas, API, `X-Leme`, token, cookie, confirmação de comandos perigosos

12.3 Integração contínua

A cada push e pull request, o GitHub Actions confere a formatação e o `go vet`, roda os testes com o detector de corridas, compila para Linux (amd64 e arm64), Windows e macOS, sobe três processos `lemed` e exercita a CLI de ponta a ponta (incluindo a queda de um nó e os códigos de saída), e constrói a imagem Docker.

Copyright (©) 2026 Erik Pimenta

13
Solução de problemas

13.1 Conexão

Mensagem	Causa provável	Solução
`não foi possível conectar a ...: connection refused`	Nó parado ou porta errada	Confira o processo e a porta; `leme ping`
`token inválido` (código 7)	Token diferente do cluster	Confira `LEME_TOKEN`, o perfil e `leme config view`
`autenticação obrigatória`	Cluster com token, CLI sem	Informe `--token` ou `--token-arquivo`
`tls: failed to verify certificate`	CA errada ou nome fora do certificado	Use a CA correta e um nome presente no `subjectAltName`
`... não respondeu a tempo` (código 4)	Rede lenta ou nó sobrecarregado	Aumente `--tempo-limite`; veja `node stats`

13.2 Dados

Mensagem	Causa provável	Solução
`quorum_nao_atingido` (código 5)	Réplicas demais fora do ar	`cluster nodes`; suba os nós ou use `-c um`
`conflito_de_versao` (código 6)	Outra escrita aconteceu antes	Leia de novo e repita com a versão nova
Chave some depois de um tempo	TTL definido	`ttl <chave>`; `expire <chave> 0` remove
`scan` lento	Muitas lápides acumuladas	`node compact todos`
Disco crescendo	Sobrescritas sem compactação	Acompanhe a fragmentação e compacte

13.3 Terminal

Sintoma	Solução
Setas aparecem como `^[[A`	O terminal não é interativo (pipe, IDE antiga); use o Windows Terminal, o PowerShell ou outro terminal

Copyright (©) 2026 Erik Pimenta

Sintoma	Solução
Acentos trocados no Windows	O Leme muda o console para UTF-8 ao abrir; em terminais antigos, rode `chcp 65001` antes
Sem cores	A saída não é um terminal, `NO_COLOR` está definido ou foi usado `--sem-cor`

Copyright (©) 2026 Erik Pimenta

14 Licenças

14.1 Componentes de terceiros

Componente	Uso	Licença
Cobra e pflag	Comandos e flags	Apache-2.0 e BSD-3-Clause
mousetrap	Dependência do Cobra no Windows	Apache-2.0
Instrument Sans, Bricolage Grotesque e IBM Plex Mono, via Fontsource	Tipografia do console e dos manuais	SIL Open Font License 1.1

Os textos das licenças das fontes acompanham os arquivos em `internal/console/web/fontes`. O nome Prometheus pertence aos seus donos e aparece só como formato de métricas compatível.

14.2 Licença do Leme

Copyright (©) 2026 Erik Pimenta

O Leme é distribuído sob a licença MIT. Você pode usar, copiar, modificar e redistribuir o código, inclusive para fins comerciais, desde que mantenha o aviso de copyright e o texto da licença, que está no arquivo `LICENSE`.